Python do Zero

Fundamentos e prática para quem está começando a programar



Tiago Sousa

PYTHON DO ZERO

Fundamentos e prática para quem está começando a programar

TIAGO SOUSA

Aviso legal

Este livro possui finalidade educacional e informativa. Seu conteúdo foi organizado para apoiar pessoas que estão começando a estudar programação e Python, oferecendo fundamentos e práticas introdutórias para a construção de pequenos programas.

A aplicação dos conceitos e das instruções apresentadas é responsabilidade do leitor. Como ferramentas, ambientes e informações técnicas podem ser atualizados ao longo do tempo, recomenda-se verificar versões e orientações atuais quando isso for necessário. Este material não substitui aconselhamento profissional específico nem garante resultados determinados em situações particulares.

Agradecimentos

Agradeço a todas as pessoas que se dedicam a aprender programação e que reconhecem o valor de começar pelos fundamentos. O interesse de quem está dando os primeiros passos é parte importante da construção de materiais claros, acessíveis e voltados à prática.

Agradeço também aos leitores que estudam, testam as instruções, observam os resultados e persistem diante das dúvidas. Cada tentativa de compreender um conceito e transformá-lo em código participa do processo de aprendizagem que este livro busca apoiar.

NAVEGAÇÃO

Sumário

Aviso legal 3

Agradecimentos 4

Introdução 6

01 · Primeiros passos com Python 8

02 · Fundamentos da linguagem 16

03 · Aplicando os conceitos na prática 25

04 · Construindo pequenos programas 33

05 · Consolidando o aprendizado 45

Conclusão 55

Introdução

Aprender a programar pode parecer difícil no início, especialmente quando surgem novos termos, regras e formas de organizar o pensamento. Python do Zero foi elaborado para apresentar os fundamentos da programação de maneira progressiva, prática, clara e acessível, acompanhando quem está dando os primeiros passos nesse universo.

Ao longo do livro, você conhecerá Python, compreenderá como as instruções são organizadas e observará como dados, comandos e resultados se relacionam. A proposta é começar pelos conceitos essenciais e avançar gradualmente, sem exigir conhecimento prévio de programação.

A aprendizagem será conduzida por meio de explicações diretas e aplicações práticas. Primeiro, você terá contato com a linguagem e com a execução de instruções simples. Em seguida, os fundamentos serão organizados para que possam ser combinados na criação de soluções cada vez mais completas.

Na parte final, o foco estará na integração do conhecimento. Você será conduzido pelo processo de planejar, organizar, construir e revisar pequenos programas, desenvolvendo mais segurança para compreender códigos e transformar necessidades simples em instruções de Python.

Este livro não pretende esgotar todos os recursos da linguagem. Seu objetivo é construir uma base sólida e compreensível para que você avance com autonomia, pratique os conceitos apresentados e continue aprendendo Python a partir dos fundamentos.

CAPÍTULO 1

Primeiros passos com Python

Conhecendo a linguagem e o ambiente de programação

Programar significa escrever instruções para que o computador realize uma tarefa. Essas instruções precisam ser organizadas de maneira clara, porque o computador não interpreta nossas intenções: ele executa aquilo que foi escrito. Neste capítulo, você conhecerá Python, entenderá como uma instrução se transforma em um resultado e executará seus primeiros comandos. A ideia não é aprender tudo de uma vez, mas começar com passos pequenos e observar o que acontece.

O que é Python?

Python é uma linguagem de programação. Uma linguagem de programação é uma forma organizada de dar instruções a um computador usando palavras, símbolos e regras que ele consegue interpretar.

Quando programamos, transformamos uma ideia em uma sequência de ações. Por exemplo, uma pessoa pode pensar “quero mostrar uma mensagem na tela”. Em Python, essa ideia pode ser escrita como uma instrução que o computador executará. O programa é, portanto, um conjunto de instruções que descreve o que deve ser

feito.

Python pode ser utilizado em diferentes tipos de trabalho, como a criação de programas, a automação de tarefas, o processamento de informações e o desenvolvimento de aplicações. Neste primeiro momento, o mais importante é perceber que a linguagem serve para transformar instruções escritas em ações executadas pelo computador.

Uma das características que tornam Python acessível para iniciantes é sua forma de escrita relativamente direta. Muitas instruções podem ser lidas com facilidade, o que ajuda a concentrar a atenção na lógica do programa. Isso não significa que não existam regras: significa que podemos começar a praticar sem precisar lidar, logo no início, com uma grande quantidade de detalhes.

Aprender Python é aprender a decompor problemas em passos. Em vez de tentar resolver uma tarefa inteira de uma só vez, você aprende a identificar uma ação, escrever uma instrução para ela, executar o código e observar o resultado. Esse ciclo será uma parte importante de sua aprendizagem.

Uma linguagem para transformar ideias em ações

Programar é transformar uma tarefa em instruções que o computador possa executar. Em Python, você começa escrevendo passos simples, executa esses passos e compara o resultado com o que

esperava obter.

Como um programa é executado

Para entender a execução de um programa, pense em uma sequência simples: você escreve uma instrução, o Python interpreta essa instrução, o computador realiza a ação correspondente e, por fim, você observa o resultado.

O código é o texto que contém as instruções do programa. Quando esse código é executado, o interpretador Python lê as instruções e procura realizar o que elas determinam. O interpretador é o componente que faz a ligação entre o código escrito por você e a execução no computador.

Uma instrução pode produzir um resultado visível, como uma mensagem na tela. Também pode realizar uma ação cujo resultado será utilizado por outra instrução. Neste capítulo, vamos trabalhar principalmente com resultados simples e visíveis, porque observá-los ajuda a verificar se o programa está funcionando como esperado.

A relação entre instrução e resultado é uma das primeiras ideias importantes da programação. Se você alterar a instrução, o resultado poderá mudar. Se escrever uma instrução incorretamente, o Python poderá apresentar uma mensagem de erro em vez de executar a ação. O erro não é uma indicação de que você não consegue pro

gramar; ele é uma informação sobre algo que precisa ser revisado.

Ao executar um comando, observe três pontos: o que foi escrito, o que apareceu como resultado e se o resultado corresponde à intenção original. Essa comparação cria uma forma prática de aprender, pois cada tentativa ajuda a compreender melhor a linguagem.

Objetivo: Representar visualmente o caminho percorrido por uma instrução simples em Python.

O diagrama deve mostrar uma sequência linear em quatro etapas, começando pela instrução escrita pelo aprendiz e terminando no resultado observado na tela.

Elementos

- Código escrito pelo aprendiz
- Interpretador Python
- Ação executada pelo computador
- Resultado observado

Relações

- O código escrito é enviado ao interpretador Python.
- O interpretador analisa a instrução e orienta a execução da ação correspondente.
- A ação executada produz um resultado que pode ser observado pelo aprendiz.
- O resultado deve ser comparado com a intenção da instrução

original.

Seu primeiro comando em Python

Uma maneira simples de exibir uma mensagem em Python é utilizar a função print. Uma função é uma instrução preparada para realizar uma tarefa. Neste caso, print solicita que um valor seja mostrado na tela.

Execute o comando a seguir em um ambiente que permita rodar Python. Pode ser o interpretador interativo, um editor de código ou outra ferramenta de execução disponível em seu ambiente.

python

```
print("Olá, mundo!")
```

print é a função utilizada para exibir algo na tela. O texto "Olá, mundo!" está entre aspas porque representa uma mensagem que deve ser tratada como texto. Ao executar o comando, o resultado esperado é a exibição de Olá, mundo!.

Aprender programando aos poucos

Depois de executar o primeiro comando, experimente fazer pequenas alterações. Troque a mensagem por outra que você queira exibir e execute novamente o código. Essa mudança é simples, mas

ajuda a perceber que o resultado depende diretamente da instrução escrita.

Observe também a diferença entre alterar o texto dentro das aspas e alterar a estrutura do comando. O texto pode ser substituído livremente por outra mensagem. Já a função print e os parênteses fazem parte da forma básica da instrução apresentada.

A prática progressiva consiste em modificar uma coisa de cada vez, executar o programa e observar o resultado. Se você mudar várias partes ao mesmo tempo, poderá ser mais difícil descobrir qual alteração causou determinado efeito. Pequenos experimentos tornam a relação entre código e resultado mais clara.

Durante a prática, compare sempre três elementos: a instrução original, a alteração realizada e o resultado produzido. Se o resultado não for o esperado, revise a escrita com calma. Verificar, corrigir e tentar novamente faz parte do processo normal de programação.

python

```
print("Estou começando a programar")
print("Python executa instruções")
```

Neste exemplo, há duas instruções print. O Python executa a primeira e depois a segunda, mostrando cada mensagem em uma linha. Para praticar, altere uma das mensagens, execute o código e

observe qual resultado mudou.

Neste capítulo, você conheceu Python como uma linguagem usada para transformar ideias em instruções executáveis. Também viu que um programa passa pelo código escrito, pela interpretação da linguagem, pela execução e pela observação do resultado. Ao utilizar print, deu os primeiros passos na escrita e na execução de comandos simples. Continue avançando de forma progressiva: escreva uma instrução, execute, observe e faça uma pequena alteração. Esse ciclo será a base para os próximos conceitos.

CAPÍTULO 2

Fundamentos da linguagem

Organizando informações e instruções

No capítulo anterior, você conheceu um primeiro comando Python e percebeu que uma instrução pode fazer o programa produzir uma saída. Agora, vamos ampliar essa ideia. Programas reais não costumam ser formados por um único comando: eles organizam várias instruções, utilizam informações e produzem resultados em uma determinada sequência. Compreender essa relação é o primeiro passo para escrever programas de forma organizada.

De comandos isolados a programas

Um comando isolado pode ser útil para experimentar uma ideia, mas um programa geralmente precisa realizar mais de uma ação. Para isso, reunimos várias instruções em uma sequência organizada. Cada instrução orienta o Python a fazer algo, como exibir uma mensagem, guardar uma informação ou realizar uma operação.

Considere, por exemplo, um programa que apresenta uma saudação e depois informa o nome de uma pessoa. A primeira instrução produz uma mensagem inicial. A segunda utiliza outra informação para produzir uma nova mensagem. As instruções estão relacionadas porque fazem parte do mesmo objetivo: apresentar uma sauda

ção completa.

Em Python, a posição das instruções no código ajuda a determinar a ordem em que elas serão executadas. Por isso, ler um programa significa observar não apenas cada linha individualmente, mas também como as linhas se conectam. Uma linha pode preparar uma informação que será usada pela linha seguinte.

A partir daqui, usaremos três ideias para analisar programas simples: dados, instruções e resultados. Dados são as informações com que o programa trabalha. Instruções são as ações que o Python executa. Resultados são os efeitos produzidos por essas ações, como uma mensagem exibida na tela.

A ideia central

Um programa é uma sequência organizada de instruções que utiliza dados para produzir resultados.

Instruções, dados e resultados

Todo programa trabalha com informações. Essas informações podem ser textos, números ou outros valores que serão utilizados durante a execução. Quando o programa precisa guardar uma informação para usá-la depois, podemos associá-la a um nome por meio de uma variável.

Uma instrução descreve uma ação. A ação pode ser guardar um valor, fazer um cálculo ou exibir algo na tela. O dado é aquilo que será utilizado pela instrução, enquanto o resultado é o que surge depois

que a instrução é executada.

Observe uma situação simples: o texto de uma saudação é um dado; o comando que solicita sua exibição é uma instrução; e a mensagem que aparece na tela é o resultado. Em uma operação matemática, os números são dados, o cálculo é realizado por uma instrução e o valor calculado é o resultado.

Essa separação ajuda a entender o funcionamento do código sem tentar decorar cada linha. Ao ler um programa, pergunte: quais informações aparecem? O que cada instrução faz com elas? O que é produzido depois? Essas perguntas formam uma maneira prática de acompanhar a execução.

A aplicação dos fundamentos deve acontecer gradualmente. Primeiro, identifique os dados. Depois, observe as instruções que os utilizam. Por fim, verifique os resultados produzidos. Com essa sequência, programas pequenos deixam de parecer uma lista de símbolos e passam a representar um processo compreensível.

Elementos básicos de um programa

Elemento	O que representa	Exemplo em uma situação simples
Dado	Uma informação utilizada pelo programa	Um nome ou um número
Instrução	Uma ação que o Python deve executar	Exibir uma mensagem ou realizar um cálculo
Resultado	O efeito produzido pela instrução	Uma mensagem na tela ou um valor calculado

A ordem das instruções importa

Por padrão, o Python executa as instruções de um programa de cima para baixo. Isso significa que a primeira instrução é processada antes da segunda, a segunda antes da terceira, e assim por diante. Essa sequência é chamada de fluxo de execução.

A ordem é importante porque algumas instruções dependem de ações anteriores. Se uma informação é guardada em uma variável, por exemplo, essa atribuição precisa acontecer antes que a informação seja utilizada. O programa precisa conhecer o valor antes de poder trabalhar com ele.

No código a seguir, a primeira linha guarda um texto na variável nome. A segunda linha utiliza essa variável para montar uma mensagem. A terceira linha exibe o resultado. Cada linha tem uma fun

ção e depende da organização estabelecida pelas linhas anteriores.

Também é possível perceber que a mudança de ordem altera o funcionamento do programa. Se a tentativa de exibir a mensagem ocorrer antes de a variável receber um valor, o programa não terá a informação necessária naquele momento. Organizar as linhas, portanto, não é apenas uma questão visual: é parte do funcionamento do programa.

python

```
nome = "Ana"
mensagem = "Olá, " + nome
print(mensagem)
```

A primeira instrução associa o texto "Ana" à variável nome. A segunda combina o texto "Olá, " com o valor guardado em nome e armazena o novo texto em mensagem. A terceira instrução exibe o conteúdo de mensagem. O resultado será a mensagem "Olá, Ana".

Leia o código como um caminho

Para entender um programa simples, acompanhe o caminho das informações: veja onde cada dado é criado ou guardado, em que instrução ele é utilizado e qual resultado aparece depois.

Lendo um pequeno programa

Agora vamos aplicar os conceitos acompanhando um programa linha por linha. O objetivo não é apenas descobrir o que aparecerá na tela, mas identificar os dados utilizados, as ações realizadas e a sequência que leva ao resultado final.

Na primeira linha, o programa guarda o número 4 na variável quantidade. Esse número é um dado. Na segunda linha, guarda o número 3 na variável preco. Na terceira linha, multiplica os dois valores e armazena o resultado na variável total. Por fim, a quarta linha exibe uma mensagem com o valor total.

A leitura pode ser feita como uma sequência de perguntas: quais dados foram armazenados? Qual instrução combinou esses dados? Em que variável o resultado foi guardado? Qual instrução tornou esse resultado visível? Essas perguntas mostram a passagem dos dados pelas instruções até o resultado.

O resultado exibido será 12, pois o programa multiplicou 4 por 3. A organização das linhas permite que cada informação esteja disponível no momento em que é necessária. Se a linha que calcula total viesse antes das linhas que definem quantidade e preco, o programa não teria esses valores preparados para realizar o cálculo.

Ao praticar, tente acompanhar cada linha sem pular diretamente para a resposta. Esse hábito ajuda a identificar erros e a compreender programas maiores. Mesmo quando o código crescer, a análise

continuará baseada nas mesmas ideias: dados, instruções, sequência e resultados.

python

```
quantidade = 4
preco = 3
total = quantidade * preco
print(total)
```

As duas primeiras linhas organizam os dados que serão utilizados. A terceira linha realiza a multiplicação e guarda o resultado em total. A última linha exibe esse resultado. A saída do programa será 12.

Roteiro para ler o programa

Pergunta	Resposta no exemplo
Quais são os dados iniciais?	quantidade vale 4 e preco vale 3
Qual ação é realizada?	Os dois valores são multiplicados
Onde o resultado é guardado?	Na variável total
O que aparece ao final?	O valor 12 é exibido

Leitura passo a passo

Ao analisar um programa, acompanhe as variáveis em cada linha. Anote mentalmente ou em papel o valor que cada uma possui depois da instrução. Essa prática torna visível a transformação dos dados até o resultado final.

Neste capítulo, você viu que um programa Python é formado por instruções organizadas em sequência. Também aprendeu a distinguir dados, instruções e resultados, além de perceber que uma instrução pode preparar informações para as próximas. Ao ler um programa simples linha por linha, você já consegue explicar não apenas o que ele produz, mas como chega a esse resultado. Essa base será utilizada nos próximos passos para criar programas cada vez mais completos.

CAPÍTULO 3

Aplicando os conceitos na prática

Transformando fundamentos em pequenos trechos de programa

Aprender os fundamentos de Python é importante, mas programar também significa combinar esses fundamentos para resolver necessidades concretas. Neste capítulo, você verá como partir de uma tarefa simples, identificar as informações necessárias, organizar instruções e conferir o resultado produzido. A ideia não é criar programas grandes, mas desenvolver um modo de pensar que permita transformar problemas pequenos em soluções claras e verificáveis.

Da necessidade ao código

Antes de escrever código, é útil entender o que precisa ser resolvido. Uma necessidade cotidiana pode ser descrita como uma pergunta: quais informações temos, o que precisamos fazer com elas e qual resultado esperamos obter? Essa análise inicial evita que as instruções sejam escritas de forma apressada ou desorganizada.

Considere a necessidade de calcular o valor total de uma compra. Para resolver essa tarefa, podemos separar o problema em partes: obter o preço, obter a quantidade, multiplicar esses valores e exibir o total. Cada parte corresponde a uma instrução ou a uma informação que o programa deverá utilizar.

Esse modo de dividir uma tarefa ajuda a transformar uma ideia geral em uma sequência de ações menores. Em Python, essas ações podem envolver variáveis para guardar informações, operadores para

realizar cálculos e a função print() para apresentar o resultado.

Uma solução simples costuma seguir uma sequência parecida com esta: entrada das informações, processamento desses dados e saída do resultado. A entrada representa aquilo que o programa recebe ou define; o processamento é o trabalho realizado sobre os dados; e a saída é aquilo que o programa mostra ao usuário.

Uma necessidade em três partes

Para analisar qualquer tarefa simples, pergunte: quais dados são necessários? Qual transformação deve ser feita? Como o resultado deverá ser apresentado? No caso de uma compra, os dados podem ser preço e quantidade, a transformação é a multiplicação e a apresentação é a exibição do valor total.

Construindo uma solução passo a passo

Depois de dividir a necessidade em partes, podemos escrever as instruções gradualmente. A construção passo a passo torna o código mais fácil de acompanhar e permite verificar cada etapa antes de avançar.

No exemplo a seguir, o preço e a quantidade são armazenados em variáveis. Em seguida, o programa calcula o total e exibe uma mensagem com o resultado. Os valores são definidos diretamente no código para manter o exemplo simples e concentrar a atenção na organização das instruções.

Uma variável funciona como um nome associado a um valor. A expressão preço * quantidade combina os valores armazenados e produz um novo resultado. Por fim, print() apresenta esse resultado na tela.

A ordem das instruções também é importante. O cálculo do total depende de preço e quantidade já existirem. Por isso, essas variáveis são definidas antes da linha que realiza a multiplicação. A mensagem só é exibida depois que o cálculo foi concluído.

python

```
preco = 12.50
quantidade = 3
total = preco * quantidade

print("Valor total:", total)
```

As duas primeiras linhas armazenam os dados da compra. A terceira linha multiplica o preço pela quantidade e guarda o resultado na variável total. A última linha exibe uma mensagem seguida do valor calculado.

Organize antes de simplificar

Um trecho de código fica mais fácil de compreender quando cada etapa possui uma função clara: guardar os dados, realizar o cálculo e mostrar o resultado. Depois que a solução estiver funcionando, será possível pensar em melhorias sem perder a organização

inicial.

Lendo e entendendo um trecho de código

Ler código é diferente de apenas observar o resultado final. Para compreender um pequeno programa, acompanhe as instruções na ordem em que Python as executa e observe como os valores são modificados ou utilizados.

No programa abaixo, a variável nome guarda um texto e a variável idade guarda um número. Em seguida, uma mensagem é montada usando esses valores e exibida na tela. A leitura pode ser feita linha por linha, perguntando o que cada instrução produz e quais dados estarão disponíveis na instrução seguinte.

Na primeira linha, nome recebe o texto Ana. Na segunda, idade recebe o número 20. Na terceira, a função print() recebe uma mensagem formada por partes separadas por vírgulas. Python exibe essas partes na mesma linha, com espaços entre elas. O resultado esperado é a mensagem Pessoa: Ana Idade: 20.

Verificar o resultado esperado antes de executar o programa cria uma previsão. Depois da execução, podemos comparar essa previsão com o que apareceu na tela. Essa comparação é uma forma simples de conferir se entendemos o código e se ele está realizando o que foi planejado.

python

```
nome = "Ana"
idade = 20

print("Pessoa:", nome, "Idade:", idade)
```

A variável nome recebe um texto, enquanto idade recebe um número inteiro. Na chamada de print(), os valores das variáveis são combinados com outros textos para formar a mensagem exibida. O programa produz: Pessoa: Ana Idade: 20.

Leia o código como uma sequência

Ao analisar um programa pequeno, pergunte em cada linha: que valor está sendo criado ou utilizado? Qual será o estado das variáveis depois desta instrução? Essa leitura sequencial ajuda a relacionar o código ao resultado observado.

Verificando se a solução funciona

Uma solução não deve ser considerada concluída apenas porque o código foi escrito. É preciso executar o programa, observar o resultado e compará-lo com aquilo que era esperado. Essa verificação pode revelar tanto erros de escrita quanto erros de raciocínio.

Comece definindo o resultado esperado com palavras ou com uma conta simples. No exemplo do preço de 12,50 e da quantidade 3, o valor esperado é 37,50. Depois, execute o programa e observe se o resultado apresentado corresponde a essa previsão.

Se o resultado estiver diferente, examine as etapas separadamente. Confira se os valores foram armazenados corretamente, se o operador utilizado representa a operação desejada e se a variável exibida é realmente aquela que contém o resultado final.

Também é importante distinguir um erro no código de um resultado inesperado causado por uma entrada diferente. Se a quantidade for alterada, por exemplo, o total deverá mudar. Verificar diferentes valores ajuda a perceber se a solução depende de um caso específico ou se funciona de acordo com a regra planejada.

A prática de prever, executar e comparar desenvolve uma leitura crítica. Com o tempo, você passará a identificar possíveis problemas antes mesmo de executar o programa, mas a execução continuará sendo uma etapa importante para confirmar o comportamento real do código.

Conferência de uma solução simples

Antes de considerar o programa pronto, verifique: a necessidade foi dividida em etapas? Todas as informações necessárias foram definidas? As instruções estão na ordem correta? O operador utilizado corresponde à operação desejada? O resultado esperado foi calculado ou descrito antes da execução? O resultado exibido corresponde ao esperado? A solução continua coerente quando um dos valores é alterado?

Prever, executar e comparar

Escolha um pequeno trecho de código já estudado. Primeiro, escreva qual resultado você espera ver. Depois, execute o programa e compare a saída com sua previsão. Se houver diferença, releia as instruções desde a primeira linha e identifique em que etapa os valores deixaram de corresponder ao que você esperava.

Transformar uma necessidade em código é um processo gradual: entender a tarefa, separar os dados, organizar as instruções, executar o programa e conferir o resultado. Ao combinar variáveis, operadores e print(), você já consegue construir pequenas soluções e desenvolver um raciocínio essencial para programar. A partir daqui, cada novo recurso de Python poderá ser aprendido como mais uma ferramenta para ampliar essas soluções, sempre mantendo a prática de dividir, testar e verificar.

CAPÍTULO 4

Construindo pequenos programas

Integrando conhecimentos em aplicações simples

Até aqui, você conheceu partes importantes da programação em Python: valores, variáveis, textos, números, operações, entrada de dados e exibição de resultados. Agora, o objetivo é juntar esses conhecimentos em um processo completo. Construir um programa não significa escrever muitas linhas de uma só vez. Uma solução confiável nasce quando você entende o problema, organiza as etapas, implementa uma parte por vez e verifica o resultado em cada momento.

Planejando antes de programar

Um pequeno programa começa antes do código. Antes de abrir o editor, defina o que a solução deverá fazer. Essa definição precisa ser simples e objetiva: qual problema será resolvido e qual resultado o usuário deverá receber?

Depois, identifique as informações necessárias. Pergunte quais dados o programa precisa receber, quais cálculos ou transformações deverá realizar e que informação deverá mostrar ao final. Essa separação ajuda a transformar uma ideia geral em etapas que podem ser programadas.

Uma forma prática de organizar o planejamento é dividir a solução em três partes: entrada, processamento e saída. A entrada reúne os dados fornecidos pelo usuário ou já disponíveis no programa. O pro

cessamento contém as operações realizadas com esses dados. A saída apresenta o resultado.

Por exemplo, um programa que calcula o valor total de uma compra pode receber o preço de um produto e a quantidade comprada, multiplicar esses valores e mostrar o total. Antes de escrever qualquer instrução, já é possível descrever o caminho da solução.

O planejamento não precisa ser um documento longo. Algumas frases ou uma lista de etapas já são suficientes para orientar a construção. O importante é imaginar o resultado e identificar uma sequência lógica antes de tentar resolver tudo diretamente no código.

Objetivo: Representar visualmente as etapas básicas do planejamento de um pequeno programa.

O diagrama deve mostrar o fluxo de uma solução simples, começando pelo objetivo e passando por entrada, processamento e saída.

Elementos

- Objetivo do programa
- Dados de entrada
- Processamento

- Resultado de saída

Relações

- O objetivo orienta a definição dos dados necessários.
- Os dados de entrada são utilizados no processamento.
- O processamento produz o resultado de saída.
- A saída deve corresponder ao objetivo definido.

Organizando a construção

Depois de planejar, construa o programa gradualmente. Em vez de escrever todas as instruções de uma vez, comece com uma versão mínima que já possa ser executada. Em seguida, acrescente novas etapas e teste novamente.

Uma sequência útil é começar pela entrada dos dados, guardar essas informações em variáveis, realizar o processamento e, por fim, apresentar a saída. Essa ordem acompanha o fluxo natural da solução e facilita a localização de problemas.

Construir progressivamente também reduz a quantidade de coisas que precisam ser verificadas ao mesmo tempo. Se o programa apresentar um resultado incorreto, você poderá observar a etapa recém-adicionada e comparar o comportamento obtido com o esperado.

Considere um programa que calcula a área de um retângulo. A primeira versão pode apenas pedir a largura e a altura. Depois, você pode acrescentar a multiplicação desses valores. Por último, pode mostrar o resultado com uma mensagem clara.

Ao integrar conceitos, observe a função de cada elemento. A função input() coleta dados, int() ou float() pode transformar esses dados em números, as variáveis guardam valores, os operadores realizam cálculos e print() comunica o resultado. Cada parte participa de uma etapa da solução.

python

```
largura = float(input("Digite a largura: "))
altura = float(input("Digite a altura: "))
area = largura * altura
print("A área do retângulo é", area)
```

O programa é construído seguindo a sequência entrada, processamento e saída. As duas primeiras linhas recebem as medidas e convertem os textos digitados em números decimais. A terceira linha calcula a área e guarda o resultado na variável area. A última linha exibe o valor calculado.

Uma etapa por vez

Uma construção progressiva torna o programa mais fácil de entender, testar e corrigir. Primeiro faça uma parte funcionar; depois

acrescente a próxima.

Um programa do início ao fim

Vamos construir um programa que calcula o valor total de uma compra a partir do nome de um produto, do preço unitário e da quantidade. O objetivo é receber essas informações e informar quanto deverá ser pago.

O planejamento pode ser escrito assim: receber o nome do produto; receber o preço de uma unidade; receber a quantidade comprada; multiplicar o preço pela quantidade; mostrar o nome do produto e o valor total.

A primeira decisão importante é escolher os tipos adequados de dados. O nome será armazenado como texto. O preço poderá ter casas decimais, por isso será convertido com float(). A quantidade será um número inteiro, convertido com int().

Observe que o código segue exatamente as etapas planejadas. Não há uma instrução escondida realizando várias tarefas ao mesmo tempo. Cada linha tem uma função clara e utiliza os conceitos estudados de maneira integrada.

Ao executar o programa, o usuário fornece os dados e acompanha o resultado. Se informar o produto como caderno, o preço como 12.50 e a quantidade como 3, o processamento multiplicará 12.50 por 3 e apresentará o total correspondente.

python

```
produto = input("Digite o nome do produto: ")
preco = float(input("Digite o preço unitário: "))
quantidade = int(input("Digite a quantidade: "))

total = preco * quantidade

print("Produto:", produto)
print("Valor total:", total)
```

O código começa coletando os três dados necessários. Em seguida, calcula o total multiplicando o preço unitário pela quantidade. Por fim, exibe o produto e o resultado. A linha em branco separa visualmente a entrada do processamento e ajuda a perceber a organização do programa.

Lendo o fluxo do programa

Para compreender o exemplo, acompanhe os valores desde a entrada até a saída. O texto digitado fica na variável produto, o preço convertido fica em preco, a quantidade convertida fica em quantidade e a multiplicação gera o valor armazenado em total. Essa leitura permite verificar se cada variável representa a informação esperada.

Objetivo: Mostrar como os dados percorrem o programa de compra até formar o resultado.

O diagrama deve apresentar três entradas que convergem para o cálculo do total e, depois, para a saída exibida ao usuário.

Elementos

- Nome do produto
- Preço unitário
- Quantidade
- Multiplicação: preço unitário × quantidade
- Nome e valor total exibidos

Relações

- O nome do produto é levado diretamente para a saída.
- O preço unitário e a quantidade são usados no cálculo.
- O cálculo produz o valor total.
- A saída reúne o nome do produto e o valor total.

Encontrando e corrigindo problemas

Um programa pode ser executado sem apresentar erro de escrita e ainda assim produzir um resultado diferente do esperado. Por isso, corrigir um programa envolve comparar duas coisas: o que deveria acontecer e o que realmente aconteceu.

Comece verificando os dados de entrada. O usuário digitou os valores esperados? O programa converteu o texto para o tipo adequado? Em seguida, examine o processamento: a operação está

correta? As variáveis usadas no cálculo contêm os valores certos? Por fim, observe se a saída mostra o resultado calculado.

Faça alterações pequenas e verificáveis. Se o problema estiver no cálculo, altere apenas essa parte e execute o programa novamente. Se estiver na conversão de um dado, corrija a conversão e teste de novo. Modificar muitas linhas ao mesmo tempo dificulta saber qual mudança resolveu ou criou o problema.

Também é útil acompanhar os valores das variáveis durante a execução. Para isso, você pode exibir temporariamente uma variável com print(). Depois de entender o problema, remova mensagens que não façam parte da saída final.

Erros básicos costumam estar relacionados a nomes de variáveis escritos de forma diferente, operações incorretas, conversões ausentes ou entradas diferentes das esperadas. A correção melhora quando você analisa o caminho completo da informação, da entrada até a saída.

Comparando o resultado esperado

Se um programa deveria calcular o total de 10 unidades a 5 reais, o resultado esperado é 50. Se aparecer 15, verifique se o código somou preço e quantidade em vez de multiplicá-los. A comparação entre expectativa e resultado aponta a etapa que precisa ser examinada.

Uma verificação simples

Execute o programa com valores fáceis de conferir mentalmente, como preço 10 e quantidade 2. Antes de executar, escreva o resultado esperado. Depois, compare com o valor exibido. Se houver diferença, examine primeiro a entrada, depois o processamento e, por último, a saída.

Construa sua própria solução

Agora é sua vez de planejar e construir um pequeno programa completo. O desafio é criar uma calculadora de média para três notas. O programa deverá receber o nome do estudante e três notas, calcular a média e mostrar o nome junto com o resultado.

Antes de escrever o código, registre o objetivo, as entradas, o processamento e a saída. As notas podem ser recebidas como números decimais. O cálculo deverá somar as três notas e dividir o resultado por 3.

Construa a solução em etapas. Primeiro, faça o programa receber e guardar os dados. Depois, implemente o cálculo. Em seguida, apresente a saída. Execute o programa com valores simples e compare o resultado mostrado com a média calculada manualmente.

Se surgir um problema, não reescreva tudo imediatamente. Verifique qual etapa apresenta diferença entre o esperado e o obtido. Observe os nomes das variáveis, as conversões e a operação matemática. Faça uma alteração por vez e teste novamente.

Uma possível estrutura inicial aparece abaixo. Ela não substitui o planejamento: use-a como referência para visualizar a organização das etapas e complete ou adapte a solução conforme o objetivo do desafio.

python

```
nome = input("Digite o nome do estudante: ")
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))
nota3 = float(input("Digite a terceira nota: "))

media = (nota1 + nota2 + nota3) / 3

print("Estudante:", nome)
print("Média:", media)
```

Este código apresenta uma solução organizada em entrada, processamento e saída. As três notas são convertidas para números decimais, somadas e divididas por 3. O desafio é executar o programa,

verificar o resultado e investigar qualquer diferença entre a média esperada e a média exibida.

Checklist da solução

Defina claramente o objetivo do programa. Liste os dados que precisam ser recebidos. Escolha nomes de variáveis que indiquem o conteúdo armazenado. Converta as notas para números antes de calcular. Separe entrada, processamento e saída. Teste com valores conhecidos. Compare o resultado obtido com o resultado esperado. Se encontrar um problema, altere uma parte por vez e execute novamente.

Construir pequenos programas é um exercício de organização, não apenas de escrita de código. Quando você define o objetivo, separa entrada, processamento e saída, implementa uma etapa por vez e compara o resultado esperado com o obtido, passa a ter um processo para resolver problemas. Esse processo permite integrar os fundamentos de Python com mais autonomia e serve como base para programas maiores e mais completos.

CAPÍTULO 5

Consolidando o aprendizado

Reunindo os fundamentos em novos programas

Ao longo deste livro, você saiu do primeiro comando em Python e começou a construir programas capazes de receber informações, realizar operações e tomar decisões. Esse caminho mostra que aprender programação não depende de memorizar comandos isolados. O aprendizado acontece quando você conecta conceitos, testa suas ideias, observa os resultados e melhora o programa aos poucos. Neste capítulo, vamos revisar essa trajetória, reunir os fundamentos em uma solução simples e definir formas de continuar praticando.

O que você já consegue fazer

No início, um programa podia ser apenas uma instrução que mostrava uma mensagem na tela. Depois, você passou a armazenar informações em variáveis, receber dados com input, transformar textos em números quando necessário, realizar operações e usar condições para escolher diferentes caminhos.

Esses recursos representam partes importantes do processo de programação. Uma variável guarda um valor que será usado depois. Uma entrada permite que o programa receba dados de quem está utilizando. Uma operação transforma ou combina informações. Uma condição faz o programa analisar uma situação e decidir o que

deve acontecer.

Quando esses elementos são usados juntos, o programa deixa de ser apenas uma sequência fixa de mensagens. Ele passa a responder aos dados recebidos. Por exemplo, um programa pode perguntar a idade de uma pessoa, verificar se ela atende a determinado critério e apresentar uma resposta adequada.

A construção de um pequeno programa normalmente segue uma sequência: compreender o problema, identificar os dados necessários, organizar as etapas, escrever o código, executar a solução, conferir o resultado e corrigir o que for necessário. Essa sequência pode ser repetida várias vezes até que o programa funcione como esperado.

Programar é conectar ideias

Um programa simples pode reunir entrada de dados, variáveis, operações e decisões. A força de cada fundamento aparece quando ele se combina com os demais para resolver um problema.

Conectando os fundamentos

Para construir uma solução organizada, comece separando o problema em perguntas menores. Que informação o programa precisa receber? Que cálculo ou transformação deve ser realizada? Existe alguma condição que muda a resposta? Qual resultado precisa ser exibido?

Considere um programa que calcula o valor final de uma compra. Ele pode receber o preço de um produto e um percentual de desconto. Em seguida, calcula o valor do desconto, subtrai esse valor do preço original e mostra o total a pagar. Cada etapa possui uma função clara e pode ser verificada separadamente.

A ordem das instruções também importa. Primeiro, o programa precisa receber os dados. Depois, pode fazer os cálculos. Por fim, deve apresentar o resultado. Se uma variável for utilizada antes de receber um valor, o programa não terá a informação necessária para continuar.

Observe como os fundamentos se complementam no exemplo a seguir. As entradas chegam como texto, por isso são convertidas para números. As variáveis dão nomes aos dados e aos resultados intermediários. Os operadores realizam os cálculos. A função print apresenta o resultado ao usuário.

python

```
preco = float(input("Digite o preço do produto: "))
desconto = float(input("Digite o percentual de desconto: "))

valor_desconto = preco * desconto / 100
valor_final = preco - valor_desconto

print("Valor do desconto:", valor_desconto)
print("Valor final:", valor_final)
```

O programa recebe o preço e o percentual de desconto, converte as entradas para números decimais e armazena os valores em variáveis. Depois, calcula o desconto e o valor final em etapas separadas. Essa separação facilita a leitura e permite conferir cada resultado durante os testes.

Prática orientada para ganhar segurança

Agora, vamos organizar uma prática completa usando o mesmo processo de construção. O objetivo é criar um programa que receba três notas, calcule a média e informe se o resultado foi suficiente para aprovação. Antes de escrever o código, descreva o problema com palavras simples.

As entradas serão três notas. O processamento será a soma dessas notas dividida por três. A decisão dependerá da comparação entre a média e um valor mínimo definido no próprio programa. A saída deverá mostrar a média e uma mensagem correspondente ao resultado.

Escrever essas etapas antes do código ajuda a evitar confusão. Também torna mais fácil descobrir se alguma informação foi esquecida. Depois de criar o programa, teste com valores que produzam resultados diferentes, como uma média acima do limite e outra abaixo dele.

Durante a conferência, não observe apenas se o programa foi executado sem apresentar erro. Verifique também se os cálculos estão corretos, se as mensagens são compreensíveis e se o comporta

mento corresponde ao problema descrito.

Da ideia ao código

Antes de programar, escreva quatro respostas: quais dados serão recebidos, quais cálculos serão feitos, qual decisão será tomada e qual resultado será mostrado. Esse pequeno planejamento transforma uma tarefa ampla em etapas concretas.

Conferência do programa

Verifique se o programa recebe todas as notas necessárias. Confirme se as entradas são convertidas para um tipo numérico. Confira se a média é calculada dividindo a soma pela quantidade correta de notas. Teste uma situação de aprovação e uma situação de não aprovação. Observe se a mensagem final corresponde ao resultado calculado.

Um novo programa para construir

Neste desafio, você criará um programa de controle simples de despesas. O programa deverá receber o valor de duas despesas, calcular o total e comparar esse total com um limite definido no código. Ao final, deverá informar o total e indicar se o limite foi respeitado ou ultrapassado.

Antes de escrever, registre suas decisões: quais variáveis serão necessárias, quais operações serão realizadas e qual condição será usada. Depois, construa o programa em pequenas partes. Primeiro,

faça as entradas funcionarem. Em seguida, calcule o total. Por fim, acrescente a decisão e as mensagens de saída.

Teste o programa com valores que mantenham o total abaixo do limite e com valores que façam o total ultrapassá-lo. Se encontrar um erro, leia a mensagem apresentada, localize a linha relacionada e compare o código com o planejamento. Corrigir erros faz parte do processo normal de programação.

Como parte do desafio, documente suas decisões em frases curtas. Explique por que escolheu determinadas variáveis, qual cálculo foi feito e qual situação a condição verifica. Essa explicação ajuda a transformar a solução em algo que você compreende, em vez de apenas um código que foi digitado.

python

```
limite = 100.0

despesa_1 = float(input("Digite o valor da primeira despesa:
"))
despesa_2 = float(input("Digite o valor da segunda despesa:
"))

total = despesa_1 + despesa_2

print("Total das despesas:", total)

if total <= limite:
    print("O limite foi respeitado.")
else:
    print("O limite foi ultrapassado.")
```

Este código apresenta uma solução de referência para o desafio. O limite é armazenado em uma variável, as duas despesas são recebidas e convertidas para números, e o total é calculado com uma soma. A estrutura condicional compara o total com o limite e escolhe uma das duas mensagens. Você pode modificar o programa depois de compreender essa versão, por exemplo, trocando as mensagens ou acrescentando novas despesas.

Próximos passos com Python

Os fundamentos estudados formam uma base, não um ponto final. A partir deles, você pode continuar praticando com programas pequenos e aumentar a complexidade gradualmente. O mais importante é escolher problemas que possam ser divididos em etapas compreensíveis.

Uma boa prática é modificar programas que você já entende. Troque os valores, altere as mensagens, acrescente uma condição ou transforme uma parte do código. Depois, tente criar uma solução semelhante sem consultar o exemplo imediatamente. Esse movimento fortalece a compreensão e a autonomia.

Também é útil manter o hábito de testar. Um programa pode funcionar em uma situação e apresentar um resultado inesperado em outra. Testar diferentes entradas ajuda a descobrir limites, corrigir erros e compreender melhor o comportamento do código.

Quando avançar, você poderá estudar novos tipos de dados, estruturas de repetição, funções, listas, organização de arquivos e outras ferramentas da linguagem. Cada assunto novo será mais fácil de compreender se você continuar usando o mesmo processo: entender o problema, dividir a solução, escrever pequenas partes, executar, testar e melhorar.

Você não precisa saber tudo para criar programas úteis. Precisa compreender os fundamentos que já conhece e praticar a combinação entre eles. A segurança cresce quando você deixa de esperar uma solução perfeita antes de começar e passa a construir, verificar e aperfeiçoar uma etapa de cada vez.

A autonomia nasce da prática organizada

Você ganha segurança não por evitar erros, mas por aprender a analisar o problema, testar o código, interpretar os resultados e fazer melhorias conscientes.

Plano para continuar aprendendo

Pratique regularmente com programas pequenos. Reescreva exemplos sem copiar todas as linhas. Modifique soluções que já compreende. Teste diferentes entradas. Anote os erros e o que aprendeu com cada correção. Escolha novos assuntos somente depois de consolidar os fundamentos atuais.

Ao concluir este capítulo, você revisou os fundamentos, observou como eles se conectam e percorreu o processo completo de construção de um pequeno programa. Continue usando essa sequência

em seus próximos estudos: compreenda, planeje, escreva, execute, teste e melhore. Cada programa concluído será uma nova oportunidade de transformar conhecimento em autonomia.

Do primeiro comando ao próximo programa

Ao longo deste livro, você partiu dos primeiros contatos com Python e com a execução de instruções simples. A partir dessa base, passou a observar como os conceitos da linguagem organizam informações, instruções e resultados dentro de um programa.

Em seguida, os fundamentos foram relacionados na prática. Compreender um conceito isoladamente é importante, mas saber combiná-lo para construir uma solução simples é o que permite transformar conhecimento em programação. A leitura de pequenos trechos de código, a verificação dos resultados e a correção de problemas fazem parte desse processo.

A construção de um pequeno programa reúne várias etapas: entender o que precisa ser feito, planejar uma solução, organizar suas instruções, executar o código e analisar o que aconteceu. Essa visão integrada ajuda a desenvolver autonomia e torna mais clara a relação entre cada fundamento e o resultado produzido.

Consolidar o aprendizado significa revisar os conceitos, praticá-los em diferentes combinações e ganhar segurança aos poucos. Não é necessário dominar tudo de uma vez. O avanço acontece quando você continua observando, testando, corrigindo e construindo soluções simples com clareza.

O primeiro comando é apenas o início. Com os fundamentos apresentados aqui, você tem uma base para continuar praticando Python, elaborar novos pequenos programas e ampliar gradualmente sua compreensão da programação.